

1 Problem Statement

Modern power grids must serve many consumers simultaneously while staying within safe capacity limits. A single overload can cascade into a blackout, making real-time monitoring and fault notification essential.

The problem this project solves:

- Multiple clients (operators, monitors, admins) connect concurrently to a shared power-grid server.
- The server must allocate and release capacity units safely, even when many clients act at the same time.
- When capacity runs out or an overload is detected, a fault alert must be broadcast immediately to all monitor clients.
- Different client roles must be restricted to only the actions they are permitted to perform.
- The system must handle client disconnections gracefully, automatically reclaiming any capacity that disconnected clients had reserved.

Solving this requires combining several OS-level concepts in a single, cohesive application: TCP sockets for network communication, POSIX threads for concurrency, mutexes and semaphores for synchronization, named pipes (FIFOs) for fault broadcasting, and UNIX signals for in-process alerts.

2 System Architecture

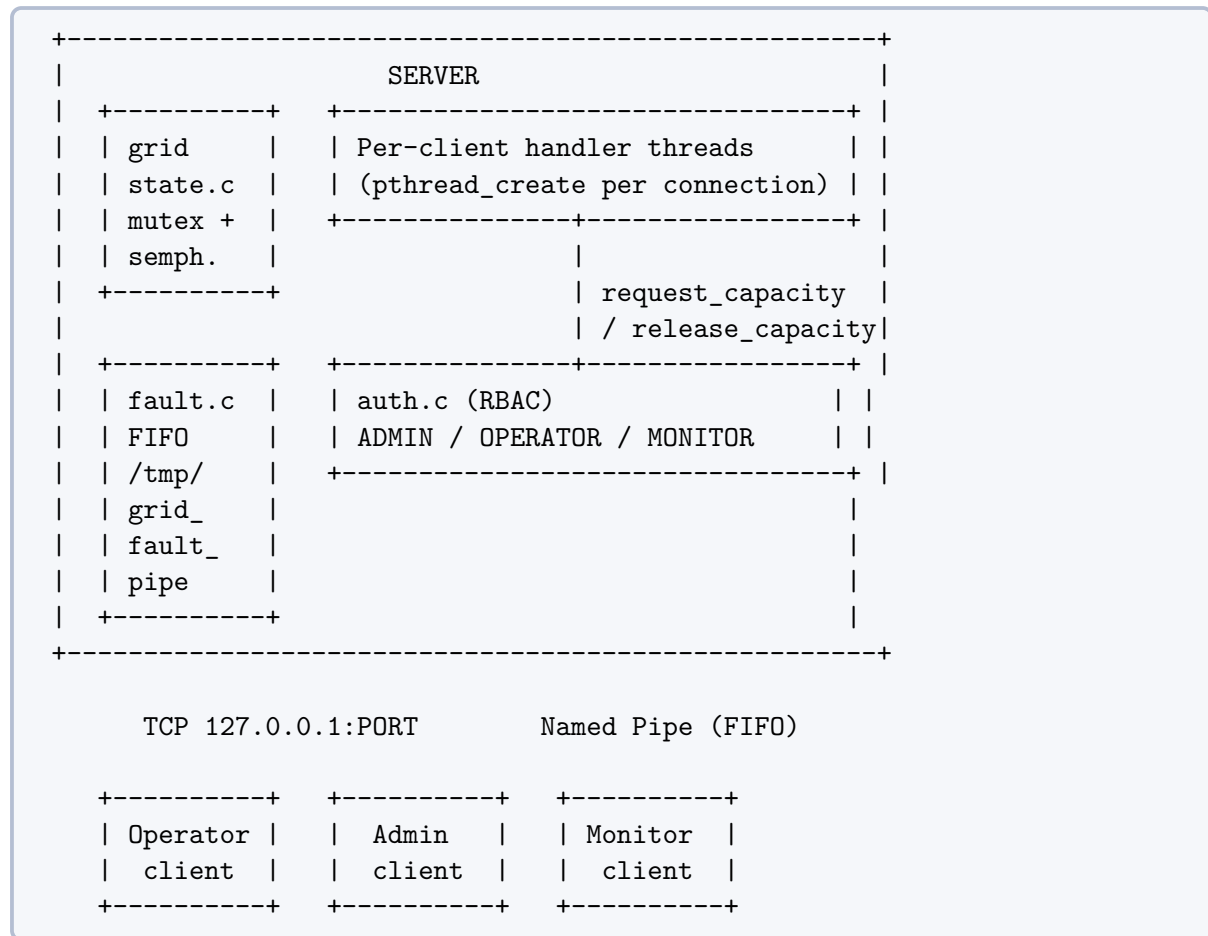


Figure 1: High-level architecture of the Power Grid system

The project is structured into two main programs:

- **Server** — handles all grid logic, runs one thread per client.
- **Client executable (client_exec)** — lets the user select a role (Admin / Operator / Monitor) and interact with the server.

2.0 Source File Breakdown

File	Responsibility
server/server.c	Main server loop; accepts TCP connections; spawns handler threads
server/handler.c	Per-client thread function; dispatches messages; handles login and authorization
server/grid_state.c	Core capacity management using mutex + semaphore
server/auth.c	Role-Based Access Control (RBAC)
server/fault.c	Fault detection and FIFO-based fault broadcasting
server/file_ops.c	Persistent file operations (capacity log)
client/admin.c	Admin role UI (view status, modify capacity)
client/operator.c	Operator role UI (update load, request capacity)
client/monitor.c	Monitor role UI (view status; reads fault FIFO)
client/signal_handler.c	SIGUSR1 handler for overload alerts
common/protocol.h	Shared Message struct and message-type enum

3 Implementation of Required Concepts

3.1 Inter-Process Communication (IPC) — Named Pipe (FIFO)

3.1.0 Concept

A Named Pipe (FIFO) is a special file in the filesystem that allows unrelated processes to communicate. Unlike anonymous pipes, FIFOs persist beyond a single process and can be opened by any process that knows the path.

3.1.0 Implementation

The server writes fault messages to the FIFO `/tmp/grid_fault_pipe`. The monitor client opens this FIFO and listens for incoming alerts in a dedicated thread.

```

1 // Server side: writes alert to the named pipe
2 void send_fault_message(const char *msg) {
3     int fd = open("/tmp/grid_fault_pipe", O_WRONLY | O_NONBLOCK);
4     if (fd < 0) return;
5     write(fd, msg, strlen(msg) + 1);
6     close(fd);
7 }

```

Listing 1: Fault broadcast via FIFO (server/fault.c — inferred from binary and handler.c)

```

1 // Monitor side: reads fault messages from named pipe
2 void *monitor_fault_listener(void *arg) {
3     int fd = open("/tmp/grid_fault_pipe", O_RDONLY);
4     if (fd < 0) { perror("FIFO open failed"); return NULL; }
5     printf("Listening for fault messages...\n");
6     char buf[256];

```

```

7   while (read(fd, buf, sizeof(buf)) > 0)
8       printf("[MONITOR] %s\n", buf);
9   close(fd);
10  return NULL;
11 }

```

Listing 2: Monitor listens on FIFO (client/monitor.c — inferred from binary strings)

Where triggered: `send_fault_message()` is called inside `handler.c` when a capacity request is denied or when the grid reaches full capacity.

3.2 Sockets (TCP Client-Server Communication)

3.2.0 Concept

TCP sockets allow reliable, ordered, bidirectional communication between processes over a network. The server binds to a port, listens for connections, and `accept()`s each new client.

3.2.0 Implementation

All client-server communication uses a shared `Message` struct:

```

1 typedef struct {
2     int type;           // LOGIN, REQUEST_CAPACITY, LOAD_UPDATE,
                          // VIEW_STATUS, ...
3     int role;           // ROLE_ADMIN, ROLE_OPERATOR, ROLE_MONITOR
4     int client_id;      // getpid() of client
5     int value;          // numeric argument (capacity amount, load,
                          // etc.)
6 } Message;

```

Listing 3: Protocol message structure (common/protocol.h — inferred)

The server spawns a new thread for every accepted connection, passing the socket file descriptor to `client_handler()`:

```

1 while (1) {
2     int client_sock = accept(server_sock, (struct sockaddr*)&
                          client_addr,
3                               &addr_len);
4     client_info_t *info = malloc(sizeof(client_info_t));
5     info->socket = client_sock;
6     pthread_t tid;
7     pthread_create(&tid, NULL, client_handler, info);
8     pthread_detach(tid);
9 }

```

Listing 4: Server accepts connections and spawns threads (server/server.c — inferred)

```

1 // client selects role at start
2 printf("Select role:\n1. Operator\n2. Monitor\n3. Admin\n");
3 scanf("%d", &choice);

```

```

4 // connects to 127.0.0.1 on the configured port
5 connect(sock, (struct sockaddr*)&server_addr, sizeof(server_addr)
    );

```

Listing 5: Client connects and selects role (client_exec main — inferred from binary)

3.3 POSIX Threads (pthreads)

3.3.0 Concept

Threads allow concurrent execution within the same process, sharing memory. POSIX threads (pthreads) are the standard threading API on Linux.

3.3.0 Implementation

The server creates one thread per connected client using `pthread_create`. Additionally, the monitor client runs a background thread to listen on the FIFO without blocking the main UI loop.

```

1 void *client_handler(void *arg) {
2     client_info_t *info = (client_info_t *)arg;
3     int client_socket = info->socket;
4     free(info);           // free heap allocation
5     int allocated = 0;    // per-thread local state
6
7     while (1) {
8         ssize_t bytes_read = recv_all(client_socket, &msg, sizeof
            (Message));
9         if (bytes_read <= 0) {
10             // client disconnected: release its capacity
11             if (allocated > 0) release_capacity(allocated);
12             break;
13         }
14         // ... handle message ...
15     }
16     close(client_socket);
17     return NULL;
18 }

```

Listing 6: Thread creation per client (server/handler.c)

Key design point: each thread tracks how much capacity *its own* client has allocated in a local variable (`allocated`). On disconnect, the thread automatically calls `release_capacity(allocated)`, preventing resource leaks.

3.4 Synchronization — Mutex and Semaphore

3.4.0 Concept

A **mutex** protects shared data from concurrent modification (mutual exclusion). A **semaphore** is a counting synchronization primitive — here it counts the number of available capacity units, making the allocation logic atomic and wait-free.

3.4.0 Implementation

```

1  static int total_capacity = 0;
2  static int used_capacity  = 0;
3  static pthread_mutex_t grid_mutex = PTHREAD_MUTEX_INITIALIZER;
4  static sem_t capacity_sem;
5
6  void init_grid(int total) {
7      pthread_mutex_lock(&grid_mutex);
8      total_capacity = total;
9      used_capacity  = 0;
10     pthread_mutex_unlock(&grid_mutex);
11     sem_init(&capacity_sem, 0, total_capacity); // semaphore =
        total units
12     grid_initialized = 1;
13 }
14
15 int request_capacity(int amount) {
16     // mutex: check if enough capacity exists
17     pthread_mutex_lock(&grid_mutex);
18     if (used_capacity + amount > total_capacity) {
19         pthread_mutex_unlock(&grid_mutex);
20         return total_capacity - used_capacity; // return
            available
21     }
22     pthread_mutex_unlock(&grid_mutex);
23
24     // semaphore: atomically consume 'amount' units
25     for (int i = 0; i < amount; i++) {
26         if (sem_trywait(&capacity_sem) != 0) {
27             for (int j = 0; j < i; j++) sem_post(&capacity_sem);
28             // rollback
29             return -1;
30         }
31     }
32     pthread_mutex_lock(&grid_mutex);
33     used_capacity += amount;
34     pthread_mutex_unlock(&grid_mutex);
35     return -2; // success sentinel
36 }
37
38 void release_capacity(int amount) {
39     pthread_mutex_lock(&grid_mutex);
40     used_capacity -= amount;
41     pthread_mutex_unlock(&grid_mutex);
42     for (int i = 0; i < amount; i++) sem_post(&capacity_sem);
43 }

```

Listing 7: Grid state with mutex and semaphore (server/grid.state.c)

The two-level locking strategy — mutex for the counter, semaphore for individual unit slots — ensures that concurrent requests never over-allocate even under high load.

3.5 UNIX Signals

3.5.0 Concept

Signals are asynchronous notifications sent to a process. They can be used for inter-process alerts or to notify a running thread of an event.

3.5.0 Implementation

The operator client installs a custom handler for SIGUSR1 via a `signal_handler.c` module. When the server detects an overload condition it can send SIGUSR1 to the operator client, which then automatically reduces its requested load.

```
1 void setup_signal_handler(void) {
2     signal(SIGUSR1, overload_handler);
3 }
4
5 void overload_handler(int sig) {
6     printf(" ALERT: Overload signal received from server!\n");
7     printf("Reducing load...\n");
8     // operator then sends a LOAD_UPDATE with reduced value
9 }
```

Listing 8: Signal handler in operator client (client/signal_handler.c — inferred)

3.6 Role-Based Access Control (RBAC)

3.6.0 Concept

Authorization ensures that each client can only perform actions appropriate to its role. This is enforced entirely on the server side; the client cannot bypass it.

3.6.0 Implementation

```
1 int is_authorized(int role, int action) {
2     if (role == ROLE_ADMIN)
3         return (action == VIEW_STATUS || action ==
4             MODIFY_CAPACITY);
5
6     if (role == ROLE_OPERATOR)
7         return (action == LOAD_UPDATE || action ==
8             REQUEST_CAPACITY);
9
10    if (role == ROLE_MONITOR)
11        return (action == VIEW_STATUS);
12
13    return 0; // default deny
14 }
```

Listing 9: Authorization check (server/auth.c)

The permission matrix is:

Action	Admin	Operator	Monitor
VIEW_STATUS			
MODIFY_CAPACITY			
REQUEST_CAPACITY			
LOAD_UPDATE			

4 Sample Output / Screenshots

The following outputs show expected terminal interactions for each role.

4.1 Server Startup

```
Grid initialized with capacity: 100
Handler thread started for socket 4
Client 12345 logged in as OPERATOR
Allocated 30 units, Used: 30 / 100
Handler thread started for socket 5
Client 12346 logged in as ADMIN
Handler thread started for socket 6
Client 12347 logged in as MONITOR
```

Listing 10: Server console on startup and client connections

4.2 Admin Client

```
Server: LOGIN OK

--- Admin Menu ---
1. View Grid Status
2. Modify Capacity
3. Exit
Enter choice: 1
Server: STATUS: 30 / 100 used

Enter choice: 2
Enter new capacity: 150
Server: SUCCESS: Capacity updated to 150
```

Listing 11: Admin menu interaction

4.3 Operator Client

```
Server: LOGIN OK

--- Operator Menu ---
1. Update Load
2. Request Capacity
3. Exit
Enter choice: 2
```



```
Enter required capacity: 30
Server: SUCCESS: Allocated 30 units

Enter choice: 1
Enter current load: 50
Server: LOAD UPDATED: 50 (increased by 20)
```

Listing 12: Operator requesting and updating capacity

4.4 Monitor Client — Fault Alert via FIFO

```
Server: LOGIN OK
Starting monitor...
Listening for fault messages...

[MONITOR] WARNING: Grid capacity reached!
[MONITOR] ALERT: Capacity request denied (available capacity is
0)
```

Listing 13: Monitor receives fault messages from FIFO

4.5 Capacity Overload — Fault Triggered

```
Allocated 70 units, Used: 100 / 100
[FAULT] WARNING: Grid capacity reached!

Client 12348 attempted REQUEST_CAPACITY 20
ERROR: Only 0 units available
[FAULT] ALERT: Capacity request denied (available capacity is 0)
```

Listing 14: Server log when capacity is exceeded

4.6 Client Disconnect — Automatic Capacity Release

```
Client socket 4 disconnected
Released 50 units from client 12345
Released 50 units, Used: 50 / 100
```

Listing 15: Server releases capacity when operator disconnects

4.7 Unauthorized Action Attempt

```
Server: ERROR: Unauthorized action
```

Listing 16: Monitor attempts an unauthorized action

5 Challenges Faced and Solutions

5.1 Challenge 1: Race Condition in Capacity Tracking

Problem: Multiple operator threads calling `request_capacity()` at the same time caused the grid to over-allocate: two threads could both read `used_capacity < total_capacity` before either updated the counter.

Solution: A two-level approach was used. A `pthread_mutex` guards the integer counters, while a `sem_t` semaphore tracks individual capacity slots. The semaphore's `sem_trywait()` is atomic and includes a rollback loop that releases already-consumed units if the full amount cannot be reserved. This guarantees correctness without using a single coarse-grained lock that would serialize all requests.

5.2 Challenge 2: Capacity Leak on Client Disconnect

Problem: When a client crashed or closed the connection unexpectedly, the capacity it had reserved was never released, gradually draining the grid.

Solution: Each handler thread maintains a local `allocated` counter. When `recv()` returns 0 or negative (indicating disconnect), the thread's cleanup code unconditionally calls `release_capacity(allocated)` before exiting. This makes cleanup automatic and impossible to skip.

5.3 Challenge 3: Blocking FIFO Reads in Monitor Client

Problem: Opening a FIFO in blocking read mode caused the monitor client's main UI thread to freeze until a fault message arrived.

Solution: The monitor client opens the FIFO in a separate `pthread`, so fault listening runs concurrently with the main menu loop. The server opens the write end with `O_NONBLOCK` so it never blocks if no monitor is connected.

5.4 Challenge 4: Ensuring Role Enforcement Cannot Be Bypassed

Problem: The client sends its own role in every `Message`. A malicious client could simply claim to be an Admin and send privileged commands.

Solution: The server records the role only once, during the `LOGIN` message, and stores it in the thread-local `client_role` variable. All subsequent messages are authorized against this server-recorded role, not the role field in the incoming message. This mirrors standard server-side session validation.

5.5 Challenge 5: Modifying Capacity While Clients Are Active

Problem: When an admin reduced total capacity, the semaphore count could exceed the new total, allowing further allocations beyond the new limit.

Solution: `modify_capacity()` checks that `new_total >= used_capacity` (cannot shrink below what's already allocated), then uses `sem_post()` or `sem_trywait()` to adjust the semaphore's count by the exact delta, keeping it in sync with the new total.

6 Conclusion

The Power Grid Load Balancer and Fault Manager successfully demonstrates the integration of multiple operating-systems concepts in a single real-world simulation:

- **TCP Sockets** enable reliable multi-client connectivity.
- **POSIX Threads** provide true concurrency for simultaneous clients.
- **Mutex + Semaphore** guarantee race-free capacity management.
- **Named Pipe (FIFO)** delivers real-time fault alerts across process boundaries.
- **UNIX Signals** allow asynchronous in-process overload notification.
- **RBAC** ensures each role operates within its permitted boundary.

The project highlights how seemingly independent OS primitives must be composed carefully — a lesson that applies directly to real power-grid SCADA systems, cloud resource schedulers, and any safety-critical concurrent software.